

LAB 4

Martim Ferreira ist1106713

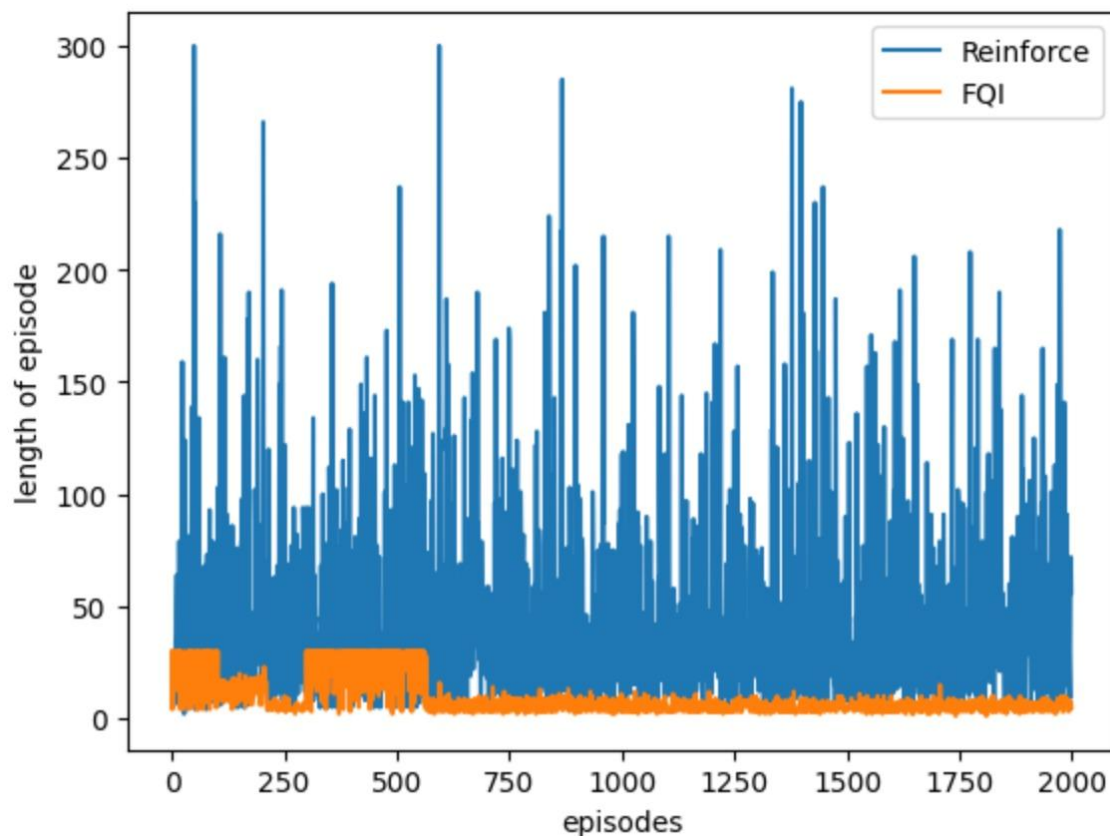
Sofia Piteira ist1106194

1-The limitations of using the discretised Q learning algorithm for this problem are essentially three: first we'd have to discretize the continuous space, that said we would have to decide a span from which the containers of the input are, making the overall accuracy of said inputs lower because we would represent various continuous points in space with only a discretizes one, which brings us to the second limitation: the curse of dimensionality, the lower we try to make the span of the containers the exponentially bigger the Q table gets, requiring more memory and computational time, and this is a problem because of the other limitation of this algorithm, because the values from any state don't directly affect the values of other neighbouring states, not generalizing information, and since the algorithm requires the program to pass through each state multiple times for it to learn an effective policy, this is not ideal.

2-The major limitation of the fitted Q-learning algorithm, as applied in this lab, is its linearity, meaning it cannot accurately represent non-linear problems like this one. This implementation uses linear regression as its main way of learning, which creates a strictly linear function approximation. Since the golf course the algorithm is trying to learn has a hole in the middle (e.g., at 200m or 400m) and not at the end, the true optimal cost-to-go resembles a 'V-shape', decreasing as the agent approaches the hole and increasing if it overshoots. A purely linear equation cannot model this shape, making it impossible for this specific function approximation to learn the optimal policy accurately.

3-Based on the cumulative costs (episode lengths) obtained during training, the FQI algorithm provided significantly better results. As shown in the graph, the FQI algorithm (orange line) goes through an initial learning phase for the first ~600 episodes where the cost occasionally hits around 30 steps. After this initial phase, it successfully converges and drops to a consistently low, stable cost (mostly under 10 steps) for the remainder of the 2000 episodes. This shows it reliably

found a highly efficient strategy. In contrast, the Reinforce algorithm (blue line) performs much worse and exhibits extreme instability. Its episode lengths are completely erratic throughout the entire training process, constantly spiking to 150, 250, and even 300 steps per episode. Even at the end of the 2000 episodes, Reinforce fails to maintain a consistently low cost and continues to show massive variance, indicating that it struggles to reliably navigate the course.



4- There are various algorithms to avoid computing a direct matrix inversion, we can use iterative numerical optimization methods such as Batch Gradient Descent (GD) or Stochastic Gradient Descent (SGD), or we can apply regularization techniques like Ridge Regression to ensure the matrix is safely invertible. For this exercise, we chose to implement a Gradient Descent (SGD) solver to iteratively update the parameters instead of using the exact least-squares solution. The Gradient Descent implementation did not improve the results. This is because our model uses a simple linear function approximation with only two features. For a linear problem of this size, the standard matrix inversion (least-squares) calculates the exact, mathematically perfect global minimum of the error in a single step. Gradient Descent is an iterative approximation that takes thousands of steps to reach the bottom of the error curve. Therefore, SGD can never find a "better"

solution than the exact optimum already found by the matrix inversion, and in practice, it trains slower and requires manual tuning of the learning rate.

5- Between the two versions of FQI (Matrix Inversion and SGD), there is actually no difference in the exploration strategy. This is because both methods use the same epsilon-greedy policy with a fixed 10% chance to choose a random action.

Therefore, both methods visit the exact same number of states and explore the environment just as uniformly as each other. The only real difference is how they process the data once it has been collected; while they see the same states, the Matrix Inversion calculates the exact best-fit weights for those states instantly, while the SGD version only approximates them through iterative steps.

5-The difference in the exploration strategy between the two methods comes down to FQI using an epsilon-greedy approach, while the Reinforce algorithm uses a Softmax policy. In our FQI implementation, the algorithm has a fixed 10% chance of taking a completely random action. This makes FQI explore more uniformly, as it will randomly select even the worst possible actions with the exact same probability throughout the entire training process. On the other hand, Reinforce converts the action weights into a probability distribution. At the beginning, it explores fairly uniformly because the weights are close to zero, but as the algorithm learns, the probability of choosing actions that lead to high costs drops significantly. Therefore, FQI explores more uniformly because its random exploration is forced and constant, whereas Reinforce explores less uniformly over time because it adapts its strategy to focus only on the more promising actions.

6- To improve the Reinforce algorithm, we modified the policy gradient update rule by removing the γ^t scaling factor. In the original implementation, the update is multiplied by γ^t , which exponentially shrinks the size of the update as the episode goes on. This creates a practical issue where if the agent takes, for example, 200 steps to finally hit the ball into the hole, the update for those final, successful steps is heavily discounted. Because of this, the agent barely learns from the specific actions that actually led to the goal. By getting rid of the γ^t term, we apply the update uniformly across all time steps, giving equal learning weight to all actions regardless of when they occurred in the episode. The results obtained with this improvement are drastically better than the original implementation. As seen in the new graph, Reinforce now converges smoothly and stably, dropping to an average cost of fewer than 10 steps per episode. This change entirely eliminates the massive variance spikes seen

previously and achieves a final performance that is highly competitive with, and arguably matches, the FQI algorithm.

